```
    sealed trait Command
    final case class AddCargo(cargo: Cargo) extends Command      ❷
      with CborSerializable                                      ❷
    final case class GetCargos(replyTo: ActorRef[List[Cargo]])   ❷
      extends Command with CborSerializable                      ❷

  def apply(containerId: String): Behavior[Command] = {          ❸
    ready(List())
  }

  def ready(cargos: List[Cargo]): Behavior[Command] = {          ❹
    Behaviors.receiveMessage[Command] {
      case AddCargo(cargo) =>
        ready(cargo +: cargos)
      case GetCargos(replyTo) =>
        replyTo ! cargos
        Behaviors.same
    }
  }
}
```

❶ Entity key that uniquely identifies the type of entity in this cluster

❷ Protocol of the actor

❸ Entity ID that makes it uniquely referrable

❹ Business logic to add or get cargos

To send a message to an entity, you must have at least one sharded cluster and initialize an SR from it. You can have one SR in a node and thus have only one type of entity, or more than one SR and different entity types. Once you have an SR, you can send a message to an entity by wrapping it in an envelope along with the entity's ID:

```
class ContainerSpec ... {

  ...

  val sharding = ClusterSharding(system)                                    ❶
  val entityDefinition = Entity(Container.typeKey)(createBehavior =
        entityContext => Container(entityContext.entityId)))               ❷

  val shardRegion: ActorRef[ShardingEnvelope[Container.Command]] =          ❸
    sharding.init(entityDefinition)

  val containerId = "id-1"
  val cargo = Container.Cargo("id-c","sack",3)

  shardRegion ! ShardingEnvelope(containerId, Container.AddCargo(cargo))   ❹

  ...
}
```

❶ Sharded cluster

❷ Entity definition

❸ Sharding region created from the key and containerId

❹ Sends the message wrapped in an envelope

It may surprise you that you don't have an `ActorRef` to send the message to—you haven't even spawned it. When you send a message to an entity that doesn't exist, the system